

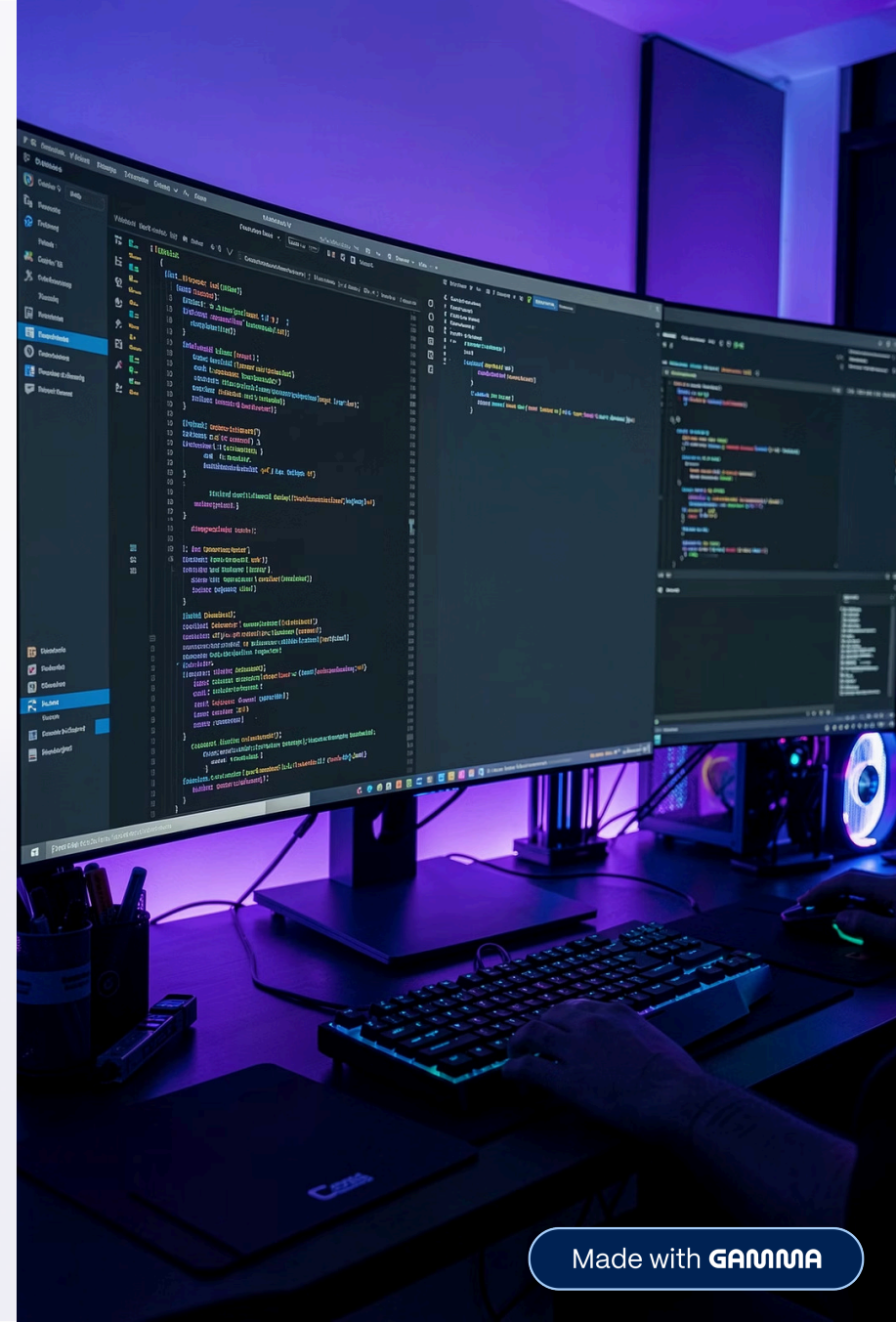
UNISUL · 2026

Sistema de Controle de Estoque

Gestão e Qualidade de Software · Prof. Jorge Werner

Curso: Análise e Desenvolvimento de Sistemas · Palhoça – SC

Isadora Luchtenberg Fernandes · Lucas Rengel · Antonio Victor Iaroseski Segala



Made with GAMMA

Objetivo

Apresentar os resultados da Atividade A3, demonstrando a evolução do software por meio de testes automatizados, integração contínua, análise estática de código e boas práticas de qualidade.

Sumário e Objetivo do Projeto

Estrutura da Apresentação



Visão Geral do Sistema

Sistema web para gestão completa de estoque: cadastro de categorias e produtos, controle de movimentações e geração de relatórios gerenciais em tempo real.

Stack Tecnológico

React

Frontend

TypeScript

Tipagem

Vite

Build

Java 17

Backend

Spring Boot

API REST

MySQL

Banco de dados

Requisitos Funcionais e Não Funcionais

Requisitos Funcionais



Cadastro de Categorias

Organização hierárquica dos itens do estoque



Cadastro de Produtos

Controle de informações e atributos dos produtos



Movimentações de Estoque

Entradas, saídas e transferências



Relatórios Gerenciais

Visão consolidada para tomada de decisão

Requisitos Não Funcionais



Segurança

Proteção de dados e acessos



Confiabilidade

Operação estável e consistente



Manutenibilidade

Código limpo e extensível



Desempenho

Respostas rápidas e eficientes



Usabilidade

Interface intuitiva e acessível

Plano de Testes

Objetivo do Plano

Definir a estratégia, os procedimentos e os critérios de verificação da qualidade do backend em Java/Spring Boot. O plano orienta a validação dos requisitos funcionais, a identificação de defeitos e a melhoria contínua da confiabilidade do sistema.

Ferramentas Utilizadas



JUnit

Testes unitários



Maven

Gerenciamento e build



JaCoCo

Cobertura de código



GitHub Actions

Pipeline CI/CD



SonarCloud

Análise estática

Objetivos Principais



Validar Requisitos

Confirmar o atendimento às regras de negócio e funcionalidades esperadas.



Prevenir Regressões

Garantir que novas alterações não comprometam comportamentos já aprovados.

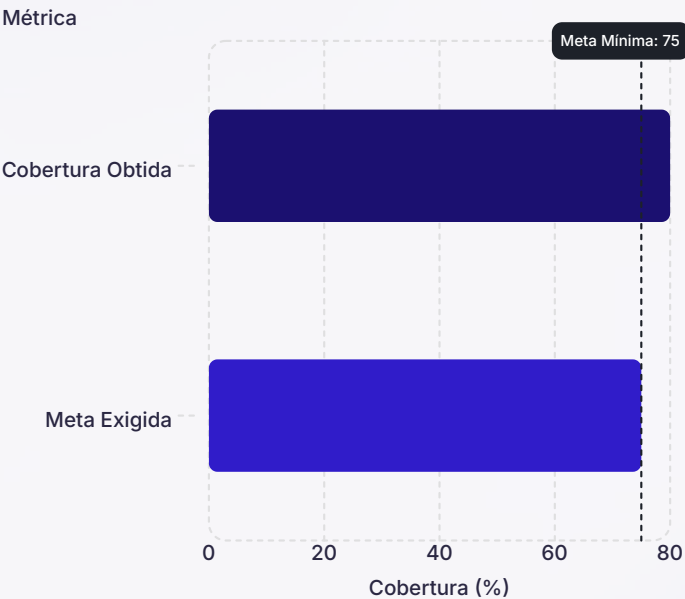


Elevar Qualidade

Aumentar a confiabilidade, a estabilidade e a robustez da aplicação.

Testes Automatizados e Cobertura de Código

Implementação de testes unitários com **JUnit** e **JaCoCo**, validando as principais regras de negócio do sistema. A meta de 75% foi superada, atingindo **80% de cobertura**.



demo

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
unisul.a3.service		86%		67%	42	91	52	454	0	27	0	4
unisul.a3.controller		61%		60%	4	29	22	67	0	24	0	4
unisul.a3.model		80%		n/a	11	91	22	156	11	91	0	10
unisul.a3.config		68%		n/a	4	6	8	36	4	6	1	3
unisul.a3		14%		n/a	1	2	7	8	1	2	0	1
Total	515 of 2,710	80%	46 of 138	66%	62	219	111	721	16	150	1	22

✔️ ☒ Meta de 75% superada — cobertura real de 80% atingida com testes unitários abrangentes.

Integração Contínua com GitHub Actions

A cada **push** ou **pull request**, o pipeline é acionado automaticamente, executando build, testes e análise de qualidade sem intervenção manual.



O pipeline garante validação automática a cada atualização do código, reduzindo erros humanos e acelerando o ciclo de desenvolvimento.

Execução Automática

Tests rodam a cada commit

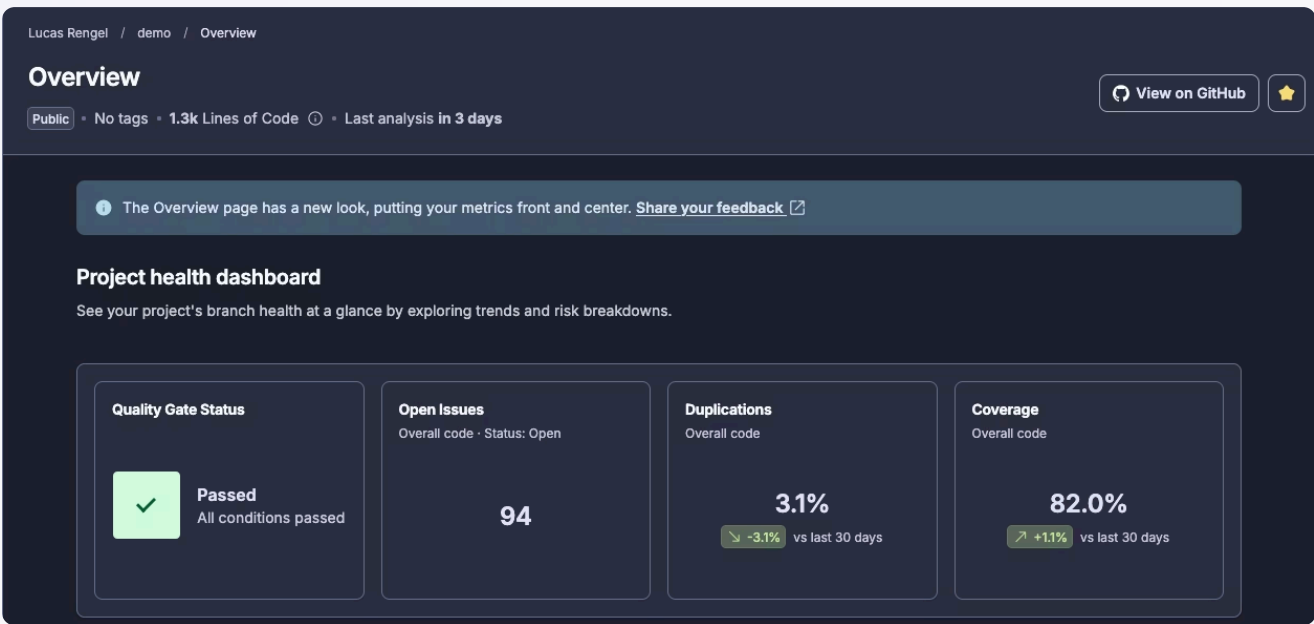
Validação Contínua

Nenhum código sem teste é mergeado

Feedback Imediato

Desenvolvedores corrigem issues rapidamente

SonarCloud e Quality Gate



Análise Estática do Código

O SonarCloud analisa automaticamente o código em busca de vulnerabilidades, bugs, code smells e duplicações, garantindo padrões de qualidade consistentes.

Segurança

Segurança monitorada pelo SonarCloud

Confiabilidade

Confiabilidade avaliada automaticamente

Manutenibilidade

Manutenibilidade acompanhada por code smells

Quality Gate

Quality Gate aprovado

Principais Issues e Evolução do Software

Antes da Implementação

 **Sem testes automatizados**

 **Sem integração contínua**

 **Sem monitoramento de qualidade**

Depois da Implementação

 **Testes automatizados com 80% de cobertura**

 **Pipeline CI/CD com GitHub Actions**

 **SonarCloud com Quality Gate aprovado**

Melhorias realizadas

- Implementação de testes unitários
- Configuração do JaCoCo
- Integração com GitHub Actions
- Integração com SonarCloud
- Correção dos problemas apontados pelo Quality Gate

Conclusão e Resultados Finais

80%

Cobertura de Testes

Meta de 75% superada

4

Ferramentas CI/CD

JUnit, JaCoCo, GitHub Actions, SonarCloud

Quality Gate Aprovado

Objetivos Alcançados

Todas as metas da A3 foram cumpridas com sucesso

Aprendizados da Equipe

Domínio de práticas modernas de qualidade e automação

Software Evoluído

Sistema mais seguro, confiável e manutenível

 A qualidade de software não é um destino, mas um processo contínuo de melhoria e aprendizado.